Fast Tree Diffusions for Probabilistic Tabular Regression via Flow Matching

Abstract

We extend Treeffuser, a score-based diffusion model for probabilistic tabular regression with gradient-boosted trees, in two directions motivated by fast sample-based probabilistic modelling. First, we evaluate conditional-mean residualization, EDM-style preconditioning, log-sigma time sampling, and noise-level features; together these reduce 95% coverage error from 0.029 to 0.026 and cut sampling time $3.1\times$ over the published baseline. Second, we replace score matching with flow matching using Gaussian probability paths, use a closed-form conditional velocity-to-score mapping, and benchmark the resulting sampler family. With a VP path and deterministic Heun ODE at five steps, the flow-matching variant ties our improved score model on CRPS skill score, reduces 95% coverage error from 0.026 to 0.017, and is a further $3.3\times$ faster. The common mechanism is better conditioning of the supervised regression problem seen by the tree ensemble. Against six fixed-grid probabilistic baselines (NGBoost, quantile-regression LightGBM, CatBoost uncertainty, iBUG, a deep ensemble, and CARD-style neural diffusion), the Treeffuser variants have the best normalized CRPS and coverage metrics under this protocol, while quantile-regression LightGBM remains strongest on raw CRPS. Stochastic ($\varepsilon > 0$) samplers do not Pareto-dominate the deterministic corner, so the empirical recommendation is the $\varepsilon = 0$ specialization.

1 INTRODUCTION

Probabilistic regression on tabular data must combine the predictive strength of gradient-boosted trees with full predictive distributions usable for calibration, decision-making, and uncertainty quantification. Treeffuser [Beltrán-Velez et al., 2024] achieves this by training LightGBM [Ke et al., 2017] regressors to estimate the score of a noise-perturbed conditional density and integrating the corresponding reverse-time stochastic differential equation (SDE) [Song et al., 2021] at inference. The framework inherits the calibration properties of score-based diffusion while avoiding neural-network density estimators.

Our notion of tractability is computational rather than likelihood-exact: the model returns Monte Carlo samples, not closed-form marginal likelihoods. The tractable object is the inference procedure itself: the final FM row uses a five-step Heun ODE, Eq. (2) converts velocities to scores in closed form when stochastic sampling is desired, and training reduces to supervised tree regression.

This paper develops Treeffuser along two axes that interact with the tree-based score regressor in ways that differ from the neural-network case. *First*, on the score-training side, we evaluate four modifications motivated by the histogram-bin density structure of tree learners. *Second*, we replace score matching with flow matching using Gaussian probability paths [Lipman et al., 2023, Liu et al., 2023], which removes the indirect score-to-drift mapping that score-matching imposes on tree predictions and opens access to the stochastic-interpolant family [Albergo et al., 2023], parameterized by a non-negative stochasticity schedule $\varepsilon(t)$ that interpolates between the deterministic ODE sampler ($\varepsilon \equiv 0$) and the score-based reverse SDE.

Contributions. We make three contributions:

1. *Score-side modifications for tree-based diffusion.* An evaluation of conditional-mean residualization, EDM preconditioning, log-σ time sampling, and an explicit log-σ feature in the histogram-split regime of LightGBM (§2.1).
2. *Flow matching with a closed-form velocity-to-score mapping.* A Gaussian-path FM trainer for Treeffuser, using Eq. (2) to recover the implied conditional score

Submitted to the 42nd Conference on Uncertainty in Artificial Intelligence (UAI 2026). **To be used for reviewing only**.

in closed form and to run stochastic samplers at any $\varepsilon(t) \geq 0$ (§2.2, Lemma 1).

3. *Regression-conditioning unification.* We identify regression conditioning for the tree ensemble—not the functional class itself—as the bottleneck behind both score- and flow-side gains, formalised through Propositions 3, 1, and 2 (§4, Appendix A).

Across ten UCI probabilistic-regression benchmarks, the score-side modifications close most of the gap between the published Treeffuser and a strong score baseline; flow matching then matches the improved score model on CRPS skill score while improving its 90/95% interval coverage error by about 35% and reducing sampling time by another 3.3×. The shared lesson is that tree-based diffusion models depend strongly on whether the noising path, parameterization, training distribution, features, and sampler make the induced supervised problem well matched to histogram splits. Stochastic samplers ($\varepsilon > 0$) do not Pareto-dominate the deterministic ODE corner on any aggregate metric, so the empirical recommendation collapses the one-parameter family to $\varepsilon = 0$.

2 METHOD

2.1 SCORE-SIDE MODIFICATIONS

The published Treeffuser uses VESDE marginals with uniform t sampling, a noise-prediction target, and a feature layout $[\, y_t,\, X,\, t\,]$ given to LightGBM. We change four pieces, each motivated by tree-based regression rather than borrowed wholesale from neural diffusion. They act on distinct levers: response/input conditioning, allocation of training rows over noise levels, explicit noise-level features, and sampler efficiency.

Conditional-mean residualization. We fit a cross-validated LightGBM mean predictor $\hat{\mu}(x)$ on the training data and apply diffusion to $y - \hat{\mu}(x)$ rather than y directly. This converts a heteroscedastic conditional density into one whose mode is approximately zero and whose remaining variation is scale only, which empirically simplifies the score-regression target. We use a high-capacity configuration (no depth cap, 63 leaves, n_{est}=300, η=0.05) chosen by a separate sweep.

EDM preconditioning. Following Karras et al. [2022], we predict a denoised target $D_\theta(y_t, t)$ scaled to unit variance across noise levels and reconstruct the score $s(y_t, t) = (D_\theta - y_t)/\sigma(t)^2$. This stabilizes the regression target for trees, which would otherwise need to extrapolate noise-prediction magnitudes across t.

Log-sigma time sampling. We draw $\log \sigma(t) \sim \mathcal{N}(-1.2, 1.2^2)$, clip to the SDE's achievable range, and invert. Trees bin features by data density, so the distribution of training t values directly controls where the score model spends capacity. This is a stronger lever than loss reweighting [Hang et al., 2023], which scales loss within fixed bins.

Log-sigma noise feature. The feature vector for LightGBM becomes $[\, y_t,\, X,\, t,\, \log \sigma(t)\,]$, giving the regressor an explicit noise-level signal aligned with EDM scaling.

2.2 FLOW MATCHING WITH GAUSSIAN PATHS

A Gaussian flow path interpolates data y_0 and a standard-normal prior z via

$$y_t = \alpha(t)\, y_0 + \beta(t)\, z, \quad t \in [0, 1], \tag{1}$$

with boundary conditions $\alpha(0) = 1$, $\alpha(1) = 0$, $\beta(0) = 0$, $\beta(1) = 1$. We compare three paths: *linear* ($\alpha = 1 - t$, $\beta = t$), *trigonometric* ($\alpha = \cos(\pi t/2)$, $\beta = \sin(\pi t/2)$), and a *VP* schedule with linear β_t giving $\alpha^2(t) = \exp(-T(t))$, $T(t) = \frac{1}{2}\beta_{\min} t + \frac{1}{4}(\beta_{\max} - \beta_{\min})t^2$. The training target is the conditional velocity $u_t = \alpha'(t) y_0 + \beta'(t) z$, regressed against the model's $v_\theta(y_t, x, t)$.

Velocity-to-score formula. For any Gaussian path with Wronskian $W(t) = \alpha(t)\beta'(t) - \alpha'(t)\beta(t)$, the conditional score is recoverable in closed form:

$$\nabla \log p_t(y_t \mid x) \;=\; \frac{\alpha'(t)\, y_t - \alpha(t)\, v_\theta(y_t, x, t)}{W(t)\, \beta(t)}. \tag{2}$$

For linear FM this reduces to $-(y_t + (1-t)v)/t$; for trig to $-y_t - (2/\pi)\cot(\pi t/2) v$.

Stochastic-interpolant family. Albergo et al. [2023] show that any non-negative $\varepsilon(t)$ defines a marginal-preserving reverse-time sampler at the population velocity/score:

$$\mathrm{d}y_\tau = \left(-v_\theta + \tfrac{\varepsilon^2}{2}\, s_\theta\right) \mathrm{d}\tau + \varepsilon\, \mathrm{d}W_\tau, \tag{3}$$

where $\tau = 1 - t$ is reverse time and s_θ is the score from (2). The choice $\varepsilon \equiv 0$ recovers the deterministic probability-flow ODE; positive $\varepsilon(t)$ broadens samples, with exact marginal preservation only for the true velocity/score. We use the schedule $\varepsilon(t) = c\, t$, which vanishes at the data endpoint where the learned score's $1/\beta(t)$ factor would otherwise dominate (see Proposition 2).

Why these choices suit trees. Three facts guide the implementation (derivations in Appendix A). First, Eq. (2) makes FM compatible with the same stochastic-interpolant samplers as score matching. Second, variance-preserving paths keep the scale of the tree input y_t nearly stationary; a linear path instead has $\mathrm{Var}(y_t) \approx (1-t)^2 + t^2$ after standardization, which compresses the middle of the path and changes split geometry. Third, $\varepsilon = 0$ is a valid member of the sampler family, while positive ε multiplies approximate-score error by the endpoint-sensitive $1/\beta(t)$ factor.

2.3 IMPLEMENTATION NOTES

The flow-matching code reuses Treeffuser's per-output-dimension LightGBM fitter and feature-construction pipeline; only the target and prior change. Sampling uses the same Heun integrator as the score path. The residualizer is unchanged across score and FM variants—a single conditional mean is used regardless of objective. At sample time, samples are inverse-transformed through the residualizer to recover the original y scale.

3 EXPERIMENTS

Datasets and protocol. Ten UCI benchmarks (Table 2) with 3 seeds and a fixed 90/10 train/test split using the full available row count for each dataset (post-split training rows range from 277 on yacht to 41,157 on protein). For each test point we draw 200 samples via Heun integration; metrics are averaged over the test set, seeds, and (for the headline) datasets.

Metrics. We report CRPS skill score $\mathrm{CRPSS} = 1 - \mathrm{CRPS}_{\text{model}}/\mathrm{CRPS}_{\text{climatology}}$, where climatology is the empirical CDF of y_{train}; mean rel-CRPS (per-dataset CRPS divided by the best variant on that dataset, then averaged); the Dawid–Sebastiani score [Dawid, 1984]; the fraction of (dataset, seed) pairs at which a Kolmogorov–Smirnov test on the probability integral transform fails to reject uniformity at $\alpha = 0.05$ ("PIT KS p>.05"); absolute coverage error at the 50/90/95% nominal interval levels [Gneiting and Raftery, 2007]; and mean sample-generation wall time per benchmark row. Timings are measured on the same Apple-arm64 workstation, exclude fitting, and include generating 200 samples per test point plus inverse transformations.

Variants compared. Three Treeffuser rows attribute contributions cleanly: (1) PUBLISHED—the original baseline with noise-prediction, uniform t, no residualization; (2) SCORE+—the baseline plus §2.1; (3) FM—VP flow matching with residualizer-C and a 5-step Heun ODE. All share the same LightGBM size. We also compare against six fixed-grid probabilistic baselines: NGBoost [Duan et al., 2020], quantile-regression LightGBM [Ke et al., 2017], CatBoost RMSE-with-uncertainty [Prokhorenkova et al., 2018], iBUG [Brophy and Lowd, 2022], a Gaussian deep ensemble [Lakshminarayanan et al., 2017], and CARD-style neural diffusion [Han et al., 2022].

3.1 HEADLINE RESULT

Table 1 reports cross-dataset means. The score-side modifications lift CRPSS from 0.652 to 0.669, raise PIT-uniformity pass rate from 0.20 to 0.60, and cut Treeffuser sample time 3.1×. Flow matching matches this CRPSS, improves score$^+$ 90/95% coverage error by about 35% each (0.037 to 0.024 and 0.026 to 0.017), and is another 3.3× faster, for a 10.2× speedup over the published Treeffuser sampler. Non-diffusion baselines sample faster still; Appendix H shows that quantile-regression LightGBM has the best raw CRPS, while the Treeffuser rows lead on normalized CRPS and coverage metrics under this protocol. Paired Wilcoxon tests across the ten datasets reject equality of score$^+$ and published CRPS at one-sided $p = 0.042$; FM trends in the same direction but does not reach 0.05 against published ($p = 0.053$); and FM versus score$^+$ is indistinguishable (two-sided $p = 0.922$).

3.2 PER-DATASET CRPSS

Table 2 shows the Treeffuser per-dataset CRPSS. The improved variants dominate the small-to-mid regime (training rows $\leq$ 11k): both beat the published baseline on every such dataset except `wine`. On the two largest datasets the picture inverts: the published baseline reclaims `california housing` and the fast FM row trails on `protein`. The win count understates the asymmetry: published wins by tight margins on raw CRPS (2–4%), whereas its losses on yacht, energy, concrete, kin8nm, and naval are far wider. This motivates dataset-adaptive residualizer selection rather than a single fixed recipe; Appendix J also shows that stochastic samplers can improve high-uncertainty tail coverage on these datasets even when they lose on aggregate CRPS.

3.3 STOCHASTICITY ABLATION

We sweep $\varepsilon \in \{0.25, 0.5, 1.0\}$ with schedule $\in \{\text{linear}, \sqrt{t}\}$ on the VP path, matched on residualizer (the C config used in the headline). Across all six SDE settings, no configuration Pareto-dominates the $\varepsilon = 0$ corner (Treeffuser-FM): CRPSS, $|\mathrm{cE}|$@90, and $|\mathrm{cE}|$@95 all degrade monotonically as ε grows, and sample time rises with the SDE solver overhead. This is consistent with Proposition 2: positive ε couples the SDE drift to a $1/\beta(t)$-amplified approximate score, and the ct schedule used here suppresses but does not remove this possible error-amplification mechanism. The closest contender (linear schedule, $\varepsilon = 0.25$) matches $|\mathrm{cE}|$@95 at 0.015 but is strictly worse on CRPSS, $|\mathrm{cE}|$@50, and KS pass rate; its timing comes from the 25-step SDE sweep rather than the full-protein headline protocol (Appendix F.1). This aggregate result should not be read as stochasticity never helping conditionally: the high-IQR diagnostic in Appendix J shows a tail-coverage tradeoff on the largest datasets. The recommendation is therefore determin-

Table 1: Headline real-data results, mean over 10 UCI benchmarks $\times$ 3 seeds. rel-CRPS is the per-dataset CRPS divided by the best displayed variant on that dataset. $|\mathrm{cE}|@q$ is absolute coverage error at the $q\%$ interval level.

Variant	CRPSS ↑	rel-CRPS ↓	DSS ↓	KS $p{>}.05$ ↑	$\lvert$cE$\rvert$@50 ↓	$\lvert$cE$\rvert$@90 ↓	$\lvert$cE$\rvert$@95 ↓	samp. time
Treeffuser-published	0.652	1.240	1.056	0.20	0.142	0.053	0.029	88.1 s
Treeffuser-score$^+$ (ours)	**0.669**	**1.047**	**−0.236**	**0.60**	0.067	0.037	0.026	28.6 s
Treeffuser-FM (ours)	**0.669**	1.048	−0.227	0.50	0.079	**0.024**	**0.017**	8.68 s
QReg-LightGBM	0.648	1.382	0.327	0.30	0.083	0.049	0.052	0.12 s
CatBoost-unc.	0.613	1.420	0.245	0.43	0.090	0.051	0.042	**0.00 s**
NGBoost	0.591	1.420	0.343	0.43	0.093	0.070	0.064	0.09 s
Deep ensemble	0.529	3.678	0.907	0.57	**0.051**	0.040	0.026	0.01 s
iBUG	0.487	3.870	1.272	0.43	0.098	0.035	0.021	1.51 s
CARD-style diffusion	0.379	6.651	1.675	0.40	0.078	0.031	0.022	0.83 s

Table 2: Per-dataset CRPSS (higher is better) and post-split training rows.

Dataset	Train rows	Published	Score$^+$	FM
yacht	277	0.944	**0.967**	0.964
energy	691	0.953	0.966	**0.966**
concrete	927	0.757	0.807	**0.809**
diabetes	400	0.120	0.126	**0.129**
wine	5847	**0.338**	0.312	0.312
kin8nm	7372	0.507	0.603	**0.612**
power	8611	0.824	**0.839**	0.839
naval	10741	0.938	0.952	**0.953**
cali. housing	18576	**0.670**	0.663	0.660
protein	41157	**0.471**	0.453	0.449

istic ODE for aggregate CRPS/latency, with stochasticity left as a conditional-calibration knob.

3.4 PATH ABLATION

Across the same ten datasets we compare linear, trigonometric, and VP paths with the same residualizer and ODE sampler. VP is the strongest on CRPS and tail coverage; trig matches VP on calibration but is mildly worse on CRPS; linear lags both on $|\mathrm{cE}|@95$ (mean 0.029 vs. 0.015). The advantage tracks variance preservation (Proposition 1): VP and trig satisfy $\alpha^2 + \beta^2 = 1$, keeping y_t at roughly unit scale across t, which stabilizes the tree regressor's feature space relative to the linear-path schedule.

3.5 NEGATIVE TRANSFERS FROM SCORE TO FM

Three score-side tricks were ported to FM and rejected by matched sweeps. Log-noise t sampling changes CRPS by $< 0.4\%$ relative to uniform-t; min-SNR loss weighting either matches uniform ($\gamma = 5$) or hurts CRPS ($\gamma = 1$); and mean-scale residualization worsens CRPS by 8–11%. The FM headline therefore keeps the simpler recipe: uniform t, uniform loss weighting, and mean residualization.

4 DISCUSSION

Two findings deserve emphasis. First, the per-row attribution table separates score-side and FM contributions cleanly, so the FM result is not implicitly absorbing improvements that belong to the score track. Second, the deterministic ODE wins under matched residualizer on the benchmark suite, despite the theoretical appeal of marginal-preserving SDE samplers. The common mechanism behind the score$^+$ and FM gains is regression conditioning for tree ensembles: EDM preconditioning stabilises the VE score path's inputs and denoising residual, while VP-FM avoids much of the same conditioning problem through the path geometry itself. This is not a single target-scaling story; log-σ sampling controls where tree capacity is spent, the log-σ feature exposes the noise regime, and Heun ODE sampling is an inference-side efficiency gain. The large-dataset inversion argues against a blanket "SDE is worse" interpretation: per-IQR-bin diagnostics show that stochasticity can improve high-uncertainty tail coverage, while the aggregate metrics penalise the same runs for broader intervals, worse CRPS, and higher latency.

Limitations. External baselines use fixed family-level grids selected on a small validation suite, not per-dataset Bayesian optimization. We include NGBoost, iBUG, CatBoost uncertainty, quantile-regression LightGBM, a deep ensemble, and a CARD-style neural diffusion baseline, but not distributional random forests [Cevid et al., 2022]. More importantly, the residualizer and score+/FM hyperparameters are fixed by sweeps run mostly on small-to-mid datasets and reused across the suite; the large-dataset inversions in Table 2 identify per-dataset residualizer and time-sampling selection as the main remaining tuning axis. Headline tables use three data/model/sampler seeds, so per-dataset gaps, especially on the largest datasets, should be read as descriptive rather than as high-powered significance estimates.

Future work. Two threads merit investigation: dataset-adaptive residualizer selection, and explicit selection criteria for stochastic-interpolant samplers when calibrated *conditional* (rather than marginal) coverage matters.

References

Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv:2303.08797*, 2023.

Nicolas Beltrán-Velez, Alessandro Antonio Grant, Alexander Feder, Cinjon Resnick, Kyunghyun Cho, and David M. Blei. Treeffuser: Probabilistic predictions via conditional diffusions with gradient-boosted trees. In *Advances in Neural Information Processing Systems*, 2024.

Jonathan Brophy and Daniel Lowd. Instance-based uncertainty estimation for gradient-boosted regression trees. In *Advances in Neural Information Processing Systems*, 2022.

Domagoj Cevid, Loris Michel, Jeffrey Naef, Nicolai Meinshausen, and Peter Bühlmann. Distributional random forests: Heterogeneity adjustment and multivariate distributional regression. *Journal of Machine Learning Research*, 23(333):1–79, 2022.

A. Philip Dawid. Statistical theory: The prequential approach. *Journal of the Royal Statistical Society. Series A*, 147(2):278–292, 1984.

Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. NGBoost: Natural gradient boosting for probabilistic prediction. In *International Conference on Machine Learning*, 2020.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

Xizewen Han, Huangjie Zheng, and Mingyuan Zhou. CARD: Classification and regression diffusion models. In *Advances in Neural Information Processing Systems*, 2022.

Tiankai Hang, Shuyang Gu, Chen Li, Jianmin Bao, Dong Chen, Han Hu, Xin Geng, and Baining Guo. Efficient diffusion training via min-SNR weighting strategy. In *International Conference on Computer Vision*, 2023.

Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, 2022.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 2017.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, 2017.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, 2018.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

A THEORETICAL DERIVATIONS AND DESIGN CONSEQUENCES

Lemma 1 (Wronskian velocity-to-score identity) *For a Gaussian path $y_t = \alpha(t)y_0 + \beta(t)z$ with conditional velocity $u_t = \alpha'(t)y_0 + \beta'(t)z$ and nonzero Wronskian $W(t) = \alpha(t)\beta'(t) - \alpha'(t)\beta(t)$, the population velocity $v(y_t, x, t) = \mathbb{E}[u_t \mid y_t, x, t]$ implies the conditional score*

$$\nabla \log p_t(y_t \mid x) = \frac{\alpha'(t)y_t - \alpha(t)v(y_t, x, t)}{W(t)\beta(t)}.$$

Proof. Solving the two linear equations for z gives

$$z = \frac{\alpha(t)u_t - \alpha'(t)y_t}{\alpha(t)\beta'(t) - \alpha'(t)\beta(t)} = \frac{\alpha(t)u_t - \alpha'(t)y_t}{W(t)}.$$

Because $p(y_t \mid y_0, x) = \mathcal{N}(\alpha y_0, \beta^2 I)$, the conditional score is $-z/\beta(t)$. Fisher's identity gives $\nabla \log p_t(y_t \mid x) = \mathbb{E}[-z/\beta(t) \mid y_t, x, t]$. Substituting the expression for z and using that y_t is fixed under the conditional expectation yields the claim. Replacing v by v_θ gives Eq. (2). □

At a fixed t with $\alpha(t) \neq 0$, the same identity inverts to $v(y_t, x, t) = \alpha'(t)y_t/\alpha(t) - W(t)\beta(t)\nabla \log p_t(y_t \mid x)/\alpha(t)$. Thus score and velocity have the same conditional structure as functions of y_t at that time; the practical differences studied here come from path-dependent scale variation across t, feature geometry, and sampler dynamics rather than an intrinsic fixed-t smoothness advantage.

Proposition 1 (Path-induced feature variance) *Let y_0, z be independent with $\mathrm{Var}(y_0) = \mathrm{Var}(z) = I$ after standardization, and let $y_t = \alpha(t)y_0 + \beta(t)z$. Then $\mathrm{Var}(y_t) = \alpha(t)^2 + \beta(t)^2$. For the three paths used in this paper:*

- *Linear ($\alpha = 1 - t$, $\beta = t$): $\mathrm{Var}(y_t) = (1-t)^2 + t^2$, with minimum $1/2$ at $t = 1/2$.*
- *Trigonometric ($\alpha = \cos(\pi t/2)$, $\beta = \sin(\pi t/2)$): $\mathrm{Var}(y_t) = 1$ for all t.*
- *VP ($\alpha^2(t) = e^{-T(t)}$, $\beta^2(t) = 1 - e^{-T(t)}$): $\mathrm{Var}(y_t) = 1$ for all t.*

Proof. Independence and unit variance give $\mathrm{Var}(y_t) = \alpha^2\mathrm{Var}(y_0) + \beta^2\mathrm{Var}(z) = \alpha^2 + \beta^2$. Substituting the three path definitions yields the listed expressions; the trig and VP identities follow from $\cos^2 + \sin^2 = 1$ and from $\alpha^2 + \beta^2 = e^{-T} + (1 - e^{-T}) = 1$. □

Consequence for tree features. Proposition 1 formalises why trig and VP paths keep the tree input near unit scale across the entire path, whereas the linear path compresses to half-variance at $t = 1/2$. For histogram-split trees this matters because the empirical distribution of y_t determines both candidate split locations and where leaf resolution is spent.

Residualization removes predictable location. If $\mu(x) = \mathbb{E}[y \mid x]$ and $r = y - \mu(x)$, then $\mathbb{E}[\|r\|^2 \mid x] = \mathrm{tr}\,\mathrm{Var}(y \mid x) \leq \mathbb{E}[\|y\|^2 \mid x]$. Diffusion or FM on residuals therefore removes a deterministic location component that would otherwise be present at every noise level. The inequality also explains the large-dataset caveat: a fixed residualizer can become a bottleneck once the inner score/velocity model has enough data to represent the mean directly.

Proposition 2 (Endpoint score-error amplification) *Let $v_\theta(y_t, x, t) = v(y_t, x, t) + \delta(y_t, x, t)$ be an approximate velocity with pointwise error δ, and let $\hat{s}_\theta, s$ denote the implied and true scores from Lemma 1. Then*

$$\hat{s}_\theta(y_t, x, t) - s(y_t, x, t) = -\frac{\alpha(t)}{W(t)\,\beta(t)}\,\delta(y_t, x, t),$$

so the pointwise score-error gain is $|\alpha(t)/(W(t)\beta(t))|$. For all three paths considered here $W(t) \to \alpha(0)\beta'(0) > 0$ is bounded near $t = 0$ while $\beta(t) \to 0$, so the gain diverges at the data endpoint.

Proof. Eq. (2) is linear in v, so

$$\hat{s}_\theta - s = \frac{\alpha'(t)y_t - \alpha(t)(v + \delta) - (\alpha'(t)y_t - \alpha(t)v)}{W(t)\beta(t)} = -\frac{\alpha(t)\,\delta}{W(t)\beta(t)}. \qquad \square$$

Endpoint amplification. Proposition 2 therefore predicts that any stochasticity schedule with $\varepsilon(t) \not\to 0$ as $t \to 0$ injects high-gain approximate score into the SDE drift; the schedule $\varepsilon(t) = ct$ used in §2.2 and the deterministic $\varepsilon \equiv 0$ corner are the two natural responses.

Proposition 3 (Population equivalence on the VE path) *For a VE path $y_t = y_0 + \beta(t)z$ with $\beta'(t) > 0$, training a normalized velocity model ν_θ against $u_t/\beta'(t) = z$ implies the same population score as VE noise prediction and VE denoising/EDM score reconstruction:*

$$s_t(y_t, x) = -\frac{\mathbb{E}[z \mid y_t, x, t]}{\beta(t)} = \frac{\mathbb{E}[y_0 \mid y_t, x, t] - y_t}{\beta(t)^2}.$$

Proof. For VE, $\alpha = 1$, $\alpha' = 0$, $u_t = \beta'(t)z$, and $W = \beta'(t)$. Lemma 1 gives $s_t = -\mathbb{E}[u_t \mid y_t, x, t]/(\beta(t)\beta'(t)) = -\mathbb{E}[z \mid y_t, x, t]/\beta(t)$. Since $y_t = y_0 + \beta(t)z$, $\mathbb{E}[y_0 \mid y_t, x, t] = y_t - \beta(t)\mathbb{E}[z \mid y_t, x, t]$, which gives the denoising form. Noise prediction estimates $-\mathbb{E}[z \mid y_t, x, t]$ and EDM reconstructs the same population denoiser after its skip/output/input preconditioning. $\square$

Proposition 3 is a population identity, not a claim that the finite LightGBM problems are identical. EDM fits the preconditioned denoising residual $F = (y_0 - c_{\mathrm{skip}}y_t)/c_{\mathrm{out}}$ and reconstructs $D_\theta = c_{\mathrm{skip}}y_t + c_{\mathrm{out}}F_\theta$, whereas normalized VE-FM would fit z directly. They imply the same score at the optimum but present different targets and feature scalings to LightGBM. This is the useful interpretation of the score$^+$–FM tie: both improve the conditioning of the regression problem seen by the tree ensemble, EDM by preconditioning the VE score objective and VP-FM by choosing a better-conditioned probability path.

Table 3: Regression-conditioning view of the main design choices. Here y_0 and z are standardized. "Conditioning issue" refers to the supervised problem fit by LightGBM, not to the population score identity.

Variant	Regressed object	Feature scale	Conditioning issue / fix
VE noise prediction	$-z$	$y_0 + \sigma z$ grows with σ	Unit target, but heteroscedastic input and $1/\sigma$ score amplification.
VE EDM score$^+$	$(y_0 - c_{\mathrm{skip}}y_t)/c_{\mathrm{out}}$	$c_{\mathrm{in}}y_t$ is near unit scale	Input and denoising residual are explicitly preconditioned.
Raw VE-FM	$\beta'(t)z$	$y_0 + \beta z$ grows with β	Velocity target inherits the schedule derivative.
Normalized VE-FM	z	$y_0 + \beta z$ unless also scaled	Population-equivalent to noise prediction after known rescaling.
VP-FM	$\alpha' y_0 + \beta' z$	$\alpha^2 + \beta^2 \approx 1$	Path keeps tree features near the standardized data scale.

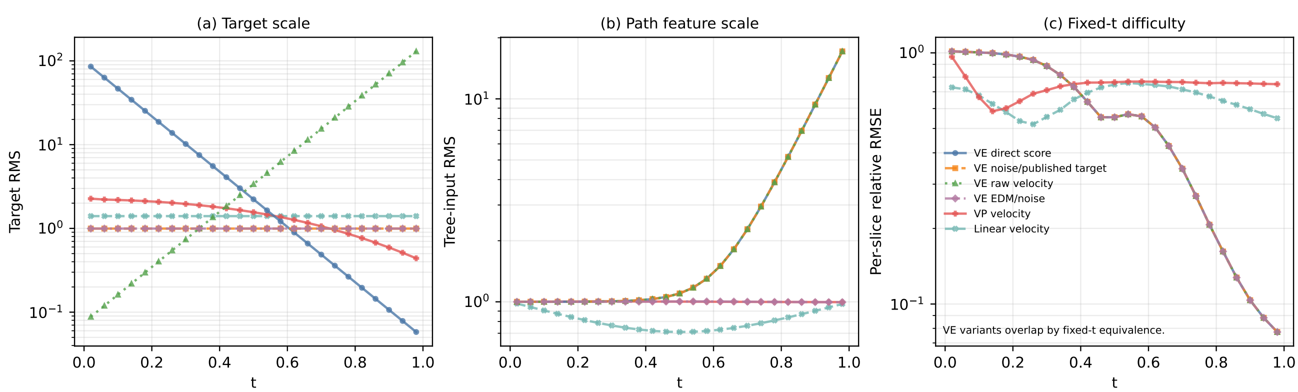

Figure 1: Toy preconditioning diagnostic on a one-dimensional heteroscedastic mixture, generated by `pixi run toy-geometry`. Panel (a) shows that direct VE score prediction and raw VE-FM have target scales that vary by orders of magnitude across t, whereas the published-style VE noise target, EDM/noise prediction, and linear velocity have unit target scale. Panel (b) separates target scaling from input geometry: the published-style VE target is stable but the tree input $y_t = y_0 + \sigma z$ still expands strongly, while EDM rescales the input and VP-FM keeps the path feature near standardized scale. Panel (c) fits a fresh LightGBM at each fixed t; the VE curves overlap up to numerical noise, confirming that the score/velocity distinction is not intrinsic fixed-t smoothness but cross-t conditioning.

B DATASETS AND PROTOCOL DETAILS

Sources. The ten UCI benchmarks follow the canonical Treeffuser protocol [Beltrán-Velez et al., 2024]: `yacht`, `concrete`, `energy`, `wine`, `kin8nm`, `naval`, `power_plant`, `protein` from the UCI repository; `california_housing` from scikit-learn's `fetch_california_housing`; and `diabetes` from scikit-learn's `load_diabetes`. All raw row counts are used before the fixed 90/10 train/test split (Table 2 reports post-split training rows); the only preprocessing is StandardScaler standardisation of features and target, fit on the training split and applied to the corresponding test split. `wine` is the union of `wine-quality-red` and `wine-quality-white` with a binary `color` indicator.

Seed policy. Seeds 0–2 (headline) and 0–9 (ablations) drive three independent streams: `data_seed` (offset 0) for the train/test split; `model_seed` (offset 10,000) for the LightGBM regressors; `sampler_seed` (offset 20,000) for the stochastic-interpolant sampler. Variance reduction across seeds therefore reflects only sampling-and-split variability, not boosting initialisation.

Common training settings. Unless noted, all Treeffuser variants use $n_{\text{estimators}} = 3000$, early stopping after 50 rounds, learning rate 0.1, $n_{\text{repeats}} = 30$ (each training point is reused with 30 fresh t samples), and the per-output-dimension LightGBM fitter. Inference draws 200 samples per test point and integrates with the Heun method.

Sampler details. TREEFFUSER-PUBLISHED uses the VESDE reverse-time SDE with the Euler–Maruyama sampler at $n_{\text{steps}} = 50$, matching the published code path. TREEFFUSER-SCORE$^+$ uses the Heun probability-flow ODE at $n_{\text{steps}} = 25$ steps under the EDM-style parametrization with $\sigma_{\min} = 0.01$, $\sigma_{\max} = 20$. TREEFFUSER-FM uses the Heun ODE at $n_{\text{steps}} = 5$ on the VP path with $\beta_{\min} = 0.1$, $\beta_{\max} = 20$.

C SCORE-SIDE MODIFICATIONS: ABLATION CROSS-REFERENCES

The four score-side modifications of §2.1 were validated by smoke and synthetic-core sweeps before being combined in the SCORE$^+$ variant. The residualizer-C configuration that the headline uses was selected by the residualizer sweep summarised in Appendix D. Log-sigma time sampling, the log-sigma noise feature, and EDM preconditioning were locked in by the `log_sigma_sweep`, `synthetic_core`, and `pf_ode_sweep` development runs. We treat those runs as model-selection evidence rather than as separately reported ablations; the quantitative claim in the paper is the combined SCORE$^+$ row in Table 1, where all four choices are evaluated together on the full ten-dataset protocol.

D RESIDUALIZER CONFIGURATION

D.1 OFF VS. MEAN RESIDUALIZATION (FM)

We isolate the contribution of mean residualization on the FM side at matched configuration (VP path, uniform t, uniform loss weighting, residualizer-C extra parameters) using ten datasets and ten seeds.

Table 4: FM with mean residualization vs. no residualization (`off`). Paired differences (mean − off, 100 paired observations); negative is better for all metrics shown.

Sampler	ΔCRPS	Δq-MACE	ΔKS stat	Δ |cE|@90	paired t on ΔCRPS
ODE @ 5 steps	−0.23	−0.022	−0.034	−0.032	−4.30
SDE @ 25 steps	−0.05	+0.001	+0.003	−0.001	−2.91

The ODE-5 advantage is large and broadly significant across calibration metrics (81/100 paired wins on q-MACE and 81/100 on |cE|@90). At SDE-25 the gap on calibration metrics is null (paired $t < 1$); the aggregate CRPS gain is driven by two real-data outliers (`yacht` +54% relative, `concrete` +15%) while six of ten datasets actually favour the unresidualized baseline. The mechanism is consistent with the few-step ODE being unable to absorb the conditional mean itself when the velocity net has ≤ 5 Heun updates; with 25 SDE updates the velocity model recovers calibration on its own. Cost: mean residualization adds ≈ 12 s of fit time on these sizes ($\sim 8\times$ vs. no residualization), so the trade is most attractive in low-step regimes.

D.2 RESIDUALIZER VARIANTS A–E (FM SIDE)

Five residualizer configurations were swept on a four-dataset, three-seed sub-suite (`california_housing`, `diabetes`, `energy`, `protein`-5k subsample) under VP-FM-ODE at 5 steps; see Table 5.

Table 5: FM residualizer ablation. "Conf. leaves" is the LightGBM `num_leaves`; "$n_{\text{est}}^{(r)}$" is the residualizer's `n_estimators`; "ES" is early-stopping rounds in the residualizer (off means no ES).

Variant	Description	Leaves	$n_{\text{est}}^{(r)}$	ES	CRPSS ↑	q-MACE ↓	|cE|@95 ↓	fit s
A	current-baseline	31	100	off	0.521	0.028	0.017	1.82
B	regularised	31	100	30	0.509	0.042	0.018	2.78
C	high capacity	63	300	off	**0.523**	0.039	0.011	8.03
D	ES moderate	63	500	30	0.516	0.033	0.021	5.91
E	ES + high capacity	63	2000	30	0.522	0.034	**0.010**	9.81

Configurations C and E sit on the calibration–latency Pareto front. C is the headline choice (best CRPSS, $1.2\times$ faster than E); E edges C on tail coverage ($|\text{cE}|@95$ 0.010 vs. 0.011). The full-protein run of §3.1 shows E reclaiming the lead on `protein` (CRPSS 0.476 vs. C's 0.449), reinforcing the case that the optimal residualizer is dataset-dependent.

D.3 SCALE-AWARE RESIDUALIZATION

The `mean_scale` residualizer additionally divides residuals by a cross-validated conditional-std estimate $\hat{\sigma}(x)$. On the ten-dataset / ten-seed protocol:

Table 6: Mean vs. mean-scale residualization on FM at matched configuration. Higher CRPSS is better; lower CRPS, DSS, q-MACE are better.

Variant	CRPSS ↑	CRPS ↓	DSS ↓	q-MACE ↓
VP-FM-ODE, mean	**0.519**	**0.652**	$+0.314$	**0.031**
VP-FM-ODE, mean-scale	0.489	0.724	$+0.665$	0.038
VP-FM-SDE, mean	**0.517**	**0.657**	$+0.380$	**0.040**
VP-FM-SDE, mean-scale	0.475	0.750	$+0.676$	0.049

Scale-aware residualization is uniformly worse: $+11\%$ CRPS on the ODE path, $+14\%$ on the SDE path, with DSS more than doubling. The per-x scale estimator is itself noisy at $n = 1000$; dividing by it injects more variance into the velocity target than it removes. `mean` is therefore the only residualizer carried into the headline.

E TIME SAMPLING AND LOSS WEIGHTING

E.1 TIME SAMPLING

Table 7: FM time-sampling sweep, 10 datasets $\times$ 3 seeds, matched residualizer-C, VP path. "uniform" is the headline choice with an endpoint anchor at $t = 1$ at probability 0.05.

Variant	CRPSS ↑	DSS ↓	q-MACE ↓	KS p>.05 ↑	samp s
VP-FM-ODE uniform	**0.6564**	-0.154	0.040	0.47	3.63
VP-FM-ODE uniform anchor16	0.6560	-0.171	0.040	0.53	3.84
VP-FM-ODE logbeta	0.6556	-0.175	**0.035**	**0.77**	3.74
VP-FM-ODE logSNR	0.6538	-0.135	0.042	0.40	3.71
score$^+$ uniform-t	0.6556	-0.210	0.038	0.50	6.95
score$^+$ log-σ t	**0.6565**	-0.169	0.038	**0.70**	6.58

For FM, log-beta and log-SNR sampling do not move CRPS by more than 0.4% relative to uniform with endpoint anchor; the smoother density shape gains $+5\%$ on KS pass rate and -13% on q-MACE but the effect is within seed variance for CRPSS. The headline keeps uniform-t for parsimony; log-beta is a viable substitute when KS uniformity is the priority. For the score model, log-sigma sampling clearly helps PIT uniformity (KS pass $0.50 \to 0.70$) at the cost of slightly worse DSS; we keep log-sigma in the SCORE$^+$ headline because the KS gain is the more interpretable calibration win.

E.2 LOSS WEIGHTING (MIN-SNR-γ)

Table 8: Min-SNR-γ loss weighting on FM, 10 datasets $\times$ 3 seeds. $\gamma = 5$ collapses to uniform on most of the support; $\gamma = 1$ amplifies the late-time loss most aggressively.

Variant	CRPSS ↑	CRPS ↓	DSS ↓	q-MACE ↓	KS p>.05 ↑
VP-FM-ODE uniform-w	**0.6564**	**4.097**	-0.159	0.040	0.47
VP-FM-ODE min-SNR-5	0.6564	4.119	-0.167	**0.037**	**0.60**
VP-FM-ODE min-SNR-1	0.6516	4.223	-0.104	0.044	0.37
VP-FM-SDE uniform-w	**0.6540**	**4.114**	-0.080	0.051	0.30
VP-FM-SDE min-SNR-5	0.6538	4.138	-0.085	0.050	0.30
VP-FM-SDE min-SNR-1	0.6485	4.239	-0.046	0.057	0.23

Min-SNR-γ is a sharpness-versus-tail trade-off. $\gamma = 1$ hurts CRPS by 0.8–1.3% across both samplers (Wilcoxon $p \leq 0.003$ for both); $\gamma = 5$ is statistically indistinguishable from uniform weighting on CRPS, but improves PIT KS pass rate on the ODE path. We keep uniform weighting in the FM headline.

F STOCHASTICITY (VELOCITY-NOISE) SWEEPS

F.1 STOCHASTICITY STRENGTH

We sweep velocity stochasticity $\varepsilon \in \{0.25, 0.50, 1.0\}$ on the VP path under both linear and $\sqrt{t}$ schedules with residualizer-C; results in Table 9 are averages over 10 datasets $\times$ 3 seeds at 25 SDE steps.

Table 9: Stochasticity sweep. "lo025" means $\varepsilon = 0.25$ etc. The $\varepsilon = 0$ corner is the headline FM (Heun ODE @ 5 steps).

Variant	CRPSS ↑	CRPS ↓	|cE|@90 ↓	|cE|@95 ↓	samp s
$\varepsilon = 0$ headline (ODE)	0.669	–	**0.024**	**0.017**	**8.7**
$\varepsilon = 0.25$ linear, res-C	0.656	4.099	0.028	0.015	5.7
$\varepsilon = 0.50$ linear, res-C	0.655	4.104	0.036	0.017	5.9
$\varepsilon = 1.00$ linear, res-C	0.654	4.114	0.047	0.022	5.9
$\varepsilon = 0.50$ sqrt, res-C	0.655	4.101	0.035	0.017	5.9
$\varepsilon = 1.00$ sqrt, res-C	0.655	4.107	0.043	0.020	5.9
$\varepsilon = 1.00$ linear, res-E	0.651	4.128	0.048	0.024	9.8

No SDE setting Pareto-dominates $\varepsilon = 0$. The closest contender ($\varepsilon = 0.25$ linear) matches |cE|@95 (0.015 vs. 0.017) but is strictly worse on CRPSS, |cE|@90, and pays 25 SDE steps vs. 5 ODE steps. Sample-time differences in the table reflect the SDE entries being measured at 25 steps without the full-protein dataset cost; the headline ODE row reflects the cross-dataset mean over the full-protein protocol.

F.2 SCHEDULE SHAPE

The schedule $\varepsilon(t) = c\,t$ (linear) is the headline form. The $\sqrt{t}$ alternative was tested for parity at $\varepsilon \in \{0.5, 1.0\}$ and produces statistically indistinguishable CRPSS (Table 9). Both schedules vanish at the data endpoint, which prevents the $1/\beta(t)$ blow-up in the velocity-to-score conversion (Eq. (2)); neither shape helps once ε is small.

G PATH CHOICE

Table 10: Flow-path ablation: linear vs. trigonometric vs. variance-preserving, 8 datasets $\times$ 3 seeds, matched residualizer and ODE sampler. Lower is better for all columns; bold is best per column.

Path	CRPS ↓	q-MACE ↓	|cE|@95 ↓	samp s
Linear	4.320	**0.026**	0.025	1.30
Trigonometric	4.317	0.030	0.017	**1.08**
Variance-preserving (VP)	**4.306**	0.029	**0.016**	1.22

VP edges trig on CRPS and tail coverage; both Pareto-dominate linear on |cE|@95. The advantage tracks variance preservation: VP and trig satisfy $\alpha^2 + \beta^2 \approx 1$, keeping y_t at roughly unit scale across t and stabilising the tree regressor's feature space. The headline uses VP.

H PROBABILISTIC-FAMILY BASELINES

We compare the headline Treeffuser variants against six external probabilistic regressors (Table 11). Hyperparameters for each baseline were selected by a fixed per-model grid summarised in this appendix's accompanying configs (`benchmarks/configs/probabilistic_baseline_hyperparams/`); this is a contextual comparison, not per-dataset Bayesian optimization. The CARD row uses a compact benchmark adapter with the same two-stage conditional-mean plus conditional-diffusion structure as CARD [Han et al., 2022].

Table 11: Probabilistic baselines: 10 UCI datasets $\times$ 3 seeds, matched full-data train/test splits. rel-CRPS is normalized by the best displayed variant on each dataset. Sample-time excludes fitting.

Model	CRPSS ↑	rel-CRPS ↓	CRPS ↓	q-MACE ↓	|cE|@95 ↓	samp s
Treeffuser-FM-residE (ours)	**0.671**	**1.050**	4.064	0.039	**0.015**	20.43
Treeffuser-FM-fast (ours)	0.669	1.061	4.060	0.039	0.017	8.68
Treeffuser-score$^+$ (ours)	0.669	1.060	4.070	**0.038**	0.026	28.57
Treeffuser-published	0.652	1.257	4.170	0.056	0.029	88.12
Quantile-regression LightGBM	0.648	1.402	**3.942**	0.041	0.052	0.12
CatBoost (uncertainty)	0.613	1.448	4.034	0.042	0.042	**0.00**
NGBoost (Gaussian)	0.591	1.446	4.257	0.046	0.064	0.09
Deep ensemble	0.529	3.742	4.588	0.042	0.026	0.01
iBUG (XGBoost)	0.487	3.963	4.699	0.061	0.021	1.51
CARD-style diffusion	0.379	6.697	6.248	0.047	0.022	0.83

Under this fixed-grid protocol, the Treeffuser variants lead on CRPSS, rel-CRPS, and coverage metrics. Quantile-regression LightGBM is strongest on raw CRPS, but it is worse on rel-CRPS and has $3\times$ the 95% coverage error of FM-FAST. Deep ensembles, iBUG, and CARD-style diffusion have reasonable tail coverage but poor normalized CRPS, indicating broad or miscentered predictive distributions rather than uniformly stronger uncertainty estimates.

I EXTENDED PER-DATASET RESULTS

Table 12 reports CRPS, |cE|@90, and sample-generation time on each of the ten benchmarks for the three headline variants, averaged over 3 seeds and over the full row count of each dataset. The "best margin" column gives the gap between the winning variant and the second-best variant in raw CRPS, which makes the asymmetry called out in §3.2 explicit: the published baseline's three wins (wine, california housing, protein) are by 2.3–3.8% while its losses on yacht / concrete / kin8nm / naval / energy are 27–69%.

FM is the fastest sampler on every dataset, with the lead widening on the largest two: on `protein` the speedup is $9.6\times$ over PUBLISHED and $4.0\times$ over SCORE$^+$; on `california_housing` the corresponding speedups are $10.5\times$ and $2.8\times$. On |cE|@90 FM leads on 7/10 datasets and SCORE$^+$ on 2/10. The per-dataset CRPS ranking reproduces the headline pattern, and the relative-margin column shows clearly that PUBLISHED's reclaim of the largest datasets is narrow whereas its losses on the small-to-mid suite are wide; the mean rel-CRPS in Table 1 assigns the published baseline a 1.240 score against 1.047 / 1.048 for SCORE$^+$ / FM.

Table 12: Extended per-dataset results, ordered roughly by n_{train}. "best margin" is winner-over-second-best in raw CRPS; the bold cell in each block is the per-metric winner on that row. Numbers come from `paper_real_data_v2_full.jsonl`.

		CRPS ↓			|cE|@90 ↓			sample s		
Dataset	best margin	Pub.	Score^{+}	FM	Pub.	Score^{+}	FM	Pub.	Score^{+}	FM
yacht	Sco^{+} +6.9%	0.330	**0.195**	0.209	0.089	0.141	**0.055**	0.5	0.3	**0.1**
diabetes	FM +0.3%	34.75	34.52	**34.41**	0.081	0.038	**0.030**	0.8	0.4	**0.1**
energy	FM +0.8%	0.259	0.190	**0.188**	0.091	0.048	**0.023**	2.1	0.5	**0.4**
concrete	FM +1.2%	2.269	1.795	**1.773**	0.077	**0.026**	0.029	1.4	0.7	**0.5**
wine	Pub +3.8%	**0.306**	0.318	0.317	0.029	0.014	**0.006**	20.7	3.6	**3.1**
kin8nm	FM +2.3%	0.076	0.061	**0.060**	0.020	**0.010**	0.012	19.6	6.7	**4.7**
power	Sco^{+} +0.4%	1.715	**1.568**	1.575	0.021	**0.012**	0.013	37.0	6.7	**4.1**
naval	FM +2.2%	0.000	0.000	**0.000**	0.089	0.071	**0.064**	185.0	24.8	**10.4**
cali. housing	Pub +2.3%	**0.207**	0.212	0.214	0.023	0.007	**0.004**	105.1	28.0	**10.0**
protein	Pub +3.4%	**1.787**	1.847	1.860	**0.005**	0.008	0.008	509.1	214.1	**53.2**

J CONDITIONAL TAIL CALIBRATION DIAGNOSTIC

The large-dataset inversion in Table 2 admits two plausible mechanisms: the fixed residualizer may be suboptimal at high n, and stochastic samplers may improve conditional tail calibration in high-uncertainty regions. Table 13 reports direct 95% coverage on `california_housing` and `protein`, including the highest predicted-IQR bin.

Table 13: High-uncertainty tail diagnostic on the two largest datasets. Cov@95 is marginal empirical coverage; Top-IQR Cov@95 is empirical coverage in the highest predicted-IQR quintile; both target 0.95. IQR-MACE is mean |cE|@95 across predicted-IQR quintiles, so lower is better. SDE-linear uses a linear stochasticity schedule; SDE-$\sqrt{t}$ uses the square-root schedule. All rows use the full training and test splits, averaged over the same three seeds as the headline run.

Dataset	Variant	CRPS	Cov@95	IQR-MACE	Top-IQR Cov@95
cali. housing	Published SDE	**0.207**	0.958	**0.015**	0.931
	FM ODE-C	0.214	**0.947**	0.021	0.900
	FM ODE-E	0.213	0.947	0.021	0.900
	FM SDE-C linear	0.215	0.969	0.023	0.939
	FM SDE-C $\sqrt{t}$	0.215	0.967	0.022	0.938
	FM SDE-E linear	0.213	0.971	0.023	**0.944**
	FM SDE-E $\sqrt{t}$	0.213	0.969	0.023	**0.944**
protein	Published SDE	1.787	0.944	0.018	0.925
	FM ODE-C	1.860	0.933	0.025	0.923
	FM ODE-E	**1.771**	**0.949**	**0.008**	0.943
	FM SDE-C linear	1.859	0.959	0.011	0.960
	FM SDE-C $\sqrt{t}$	1.856	0.956	0.012	**0.954**
	FM SDE-E linear	1.779	0.974	0.024	0.973
	FM SDE-E $\sqrt{t}$	1.776	0.971	0.021	0.971

The diagnostic supports a tradeoff rather than a one-sided conclusion. On `california_housing`, the deterministic FM rows under-cover the high-IQR bin (0.900), while adding FM stochasticity lifts this coverage to 0.938–0.944; the price is marginal over-coverage (0.967–0.971) and no gain over the published SDE on CRPS or IQR-MACE. On `protein`, residualizer-E with the deterministic ODE is already close to the nominal target and remains best on CRPS and IQR-MACE, whereas the residualizer-E SDE rows over-cover both marginally and in the high-IQR bin. Residualizer-C stochasticity is directionally useful for coverage, especially with the square-root schedule, but gives up CRPS. Thus the safest interpretation is not that SDE samplers are dominated, but that their benefit is conditional and must be selected against conditional-calibration metrics rather than headline CRPS alone.

K REPRODUCIBILITY

All sweeps in this paper are driven by YAML configs under `benchmarks/configs/`. Each result file is a JSON Lines document recording the full hyperparameter set, the random seeds (data/model/sampler), the git commit hash of the model code, a hash of the Treeffuser source tree, and every reported metric on a per-(dataset, seed) row. The full-data Treeffuser and original external-baseline runs used commit `99e3665` of the model source (`git_sha` field on every row of `paper_real_data_v2_full.jsonl` and `paper_probabilistic_baselines_selected_full.jsonl`); the CARD add-on rows are recorded separately in `paper_probabilistic_baselines_card_selected_full.jsonl`. The configs that produced the appendix tables are listed alongside in Table 14.

Table 14: Provenance: appendix tables and the config / result file each was produced from.

Table	Config	Result file
4	fm_off_vs_mean_sweep.yaml	fm_off_vs_mean_sweep__*.jsonl
3	analytic summary	–
5	residualizer_fm_sweep.yaml	residualizer_fm_sweep__*.jsonl
6	fm_mean_scale_sweep.yaml	fm_mean_scale_sweep__*.jsonl
7	fm_t_sampling_sweep.yaml	fm_t_sampling_sweep__*.jsonl
8	fm_loss_weighting_sweep.yaml	fm_loss_weighting_sweep__*.jsonl
9	sde_lower_stoch_sweep.yaml, stochasticity_schedule_sweep.yaml	sde_lower_stoch_sweep__*.jsonl, stochasticity_schedule_sweep__*.jsonl
10	flow_path_sweep.yaml	flow_path_sweep__*.jsonl
11	paper_probabilistic_baselines_selected.yaml	paper_probabilistic_baselines_selected_full.jsonl, paper_probabilistic_baselines_card_selected_full.jsonl
2, 12, 1	paper_real_data_v2.yaml	paper_real_data_v2_full.jsonl
13	paper_real_data_v2.yaml, tail_calibration_full.yaml	paper_real_data_v2_full.jsonl, tail_calibration_full.jsonl